# Manejo de Datos

Facultad de Ciencias, UNAM — Tarea 02: Listas, Expresiones Regulares, Pilas y Colas

## 1. Tabla de Complejidades

| # | Nombre del Problema | Estructura Principal | Tiempo | Espacio |
|---|---|---|---|---|
| 1 | Two Sum | Diccionario (Hash Map) | `O(n)` | `O(n)` |
| 238 | Product of Array Except Self | Listas (Arreglos) | `O(n)` | `O(1)` |
| 929 | Unique Email Addresses | Conjuntos (Set), Regex | `O(n × m)` | `O(n × m)` |
| 592 | Fraction Addition | Regex, Variables | `O(n)` | `O(n)` |
| 20 | Valid Parentheses | Pila (Lista) | `O(n)` | `O(n)` |
| 150 | Evaluate Reverse Polish Not. | Pila (Lista) | `O(n)` | `O(n)` |
| 232 | Implement Queue using Stacks | Pilas (Listas) | `O(1)` amortizado | `O(n)` |
| 622 | Design Circular Queue | Lista estática (Fija) | `O(1)` | `O(k)` |
| 65 | Valid Number | Regex ( `re.fullmatch` ) | `O(n)` | `O(1)` |
| 239 | Sliding Window Maximum | Cola Monótona ( `deque` ) | `O(n)` | `O(k)` |
| 739 | Daily Temperatures | Pila Monótona (Lista) | `O(n)` | `O(n)` |
| 56 | Merge Intervals (Extra) | Lista (Arreglo dinámico) | `O(n log n)` | `O(n)` |

*Nota: En el problema 929, $m$ representa la longitud promedio de los correos. En los problemas 622 y 239, $k$ representa el tamaño definido para la estructura o ventana.

## 2. Reflexión

El rendimiento de las colecciones integradas de Python depende directamente de su estructura subyacente. Usar `list.pop(0)` presenta una complejidad temporal de `O(n)` porque las listas en Python están implementadas a nivel de memoria en C como arreglos dinámicos contiguos. Al eliminar el primer elemento, todos los elementos subsecuentes ( `n-1` ) deben desplazarse obligatoriamente una posición hacia la izquierda en la memoria para rellenar el hueco que quedó, lo cual toma un tiempo lineal. En contraste directo, `deque.popleft()` opera de manera constante en `O(1)` porque un `deque` (double-ended queue) está implementado internamente como una lista doblemente enlazada de bloques de memoria; eliminar un elemento del frente únicamente requiere ajustar los punteros del nodo inicial, sin tener que desplazar el resto de los elementos guardados.

En nuestro caso, el problema que más trabajo nos costó estructurar adecuadamente fue "Design Circular Queue" (#622). Aunque el concepto teórico es simple, gestionar los punteros circulares *head* y *tail* junto con la variable *size* mediante aritmética modular ( `% k` ) requirió mucha atención para evitar desbordamientos y manejar correctamente los casos límite donde la cola se llena o se vacía repetidamente. Inicialmente pensamos en simplemente mover un índice general libremente, pero mantener un arreglo fijo estático para cumplir con el espíritu del problema y lograr verdaderamente la eficiencia de `O(1)` estricta requirió sincronizar múltiples estados internos simultáneamente y de forma precisa en cada método.

Por último, en el contexto de expresiones regulares, la ventaja principal de usar `re.fullmatch` sobre `re.match` es su rigurosidad a la hora de validar formatos completos. `re.match` solo comprueba que el patrón solicitado coincida al inicio de la cadena, lo cual puede dar falsos positivos si hay información sobrante o incorrecta al final. Por ejemplo, al validar un correo electrónico simple con el patrón `r'\w+@\w+\.com'` , una cadena como *"usuario@gmail.com ESTO ES BASURA"* pasaría la validación de `re.match` porque encuentra la estructura del correo al principio. Sin embargo, `re.fullmatch` requerirá que toda la cadena completa de principio a fin encaje exactamente en el patrón, por lo que fallará correctamente en este ejemplo, asegurando que los datos introducidos sean exactamente lo esperado y nada más.